

Comez Webhooks

Client Integration Guide — receive real-time store events into your CRM, ERP, Zapier, or custom backend

Contents

- 1. Quick start
- 2. Supported events
- 3. HTTP request from Comez
- 4. Event payloads
- 5. Verify the signature (required)
- 6. Your endpoint requirements
- 7. Retries and reliability
- 8. Security checklist
- 9. Testing
- 10. FAQ

Comez sends an **HTTPS POST** to your endpoint whenever a subscribed event occurs. Delivery is asynchronous and retried on failure.

1. Quick start

- 1. In the Comez admin (Webhooks), choose an **event** and paste your **HTTPS endpoint URL**.
- 2. On create, Comez shows a **signing secret once** — save it immediately. It will not be shown again.
- 3. On your server, verify every request using `X-Comez-Signature` (see below).
- 4. Respond with **HTTP 2xx** within **5 seconds**.

2. Supported events

Event	When it is sent
<code>order.received</code>	After an order is successfully placed. COD: when the order is created. Online payments: after payment is verified.
<code>cart.abandoned</code>	When an abandoned cart is detected (inactive cart or failed online checkout).

3. HTTP request from Comez

```
POST https://your-domain.com/webhooks/comez
Content-Type: application/json
X-Comez-Event: order.received
X-Comez-Signature: sha256=<hmac_hex>
X-Comez-Delivery-Id: <uuid>
User-Agent: Comez-Webhooks/1.0
```

Headers

Header	Description
<code>Content-Type</code>	Always <code>application/json</code>
<code>X-Comez-Event</code>	Event name (<code>order.received</code> or <code>cart.abandoned</code>)
<code>X-Comez-Signature</code>	HMAC-SHA256 of the raw body, prefixed with <code>sha256=</code>
<code>X-Comez-Delivery-Id</code>	Unique ID for this delivery attempt (use for idempotency)
<code>User-Agent</code>	<code>Comez-Webhooks/1.0</code>

Envelope (all events)

```
{
  "event": "order.received",
  "delivery_id": "52edd268-fef0-43b6-9cee-c179b0e99f23",
  "created_at": "2026-07-15T12:24:26.634Z",
  "data": { }
}
```

Field	Type	Description
<code>event</code>	string	Same as <code>X-Comez-Event</code>
<code>delivery_id</code>	string (UUID)	Delivery identifier
<code>created_at</code>	string (ISO 8601)	When Comez built this payload
<code>data</code>	object	Event-specific payload

4. Event payloads

`order.received` — data

```
{
  "order_id": "A8059",
  "payment_status": "sucssess",
  "payment_type": "online",
  "total": "130",
  "final_amount": "99",
  "shipping_price": "60",
  "total_discounted": "31",
  "customer": {
    "id": null,
    "name": "Maddy",
    "email": "maddy@gmail.com",
    "phone": "8056880490"
  },
  "items": [
    {
      "product_id": "102",
      "product_name": "Muslin Langot 4 Layer Padding 0-3m (Panda)",
      "variant_id": "1506",
      "variant_name": "0-3m-Panda",
      "quantity": "1",
      "price": "130",
      "net_amount": "130"
    }
  ],
  "created_at": "2026-07-15T12:20:26.446Z"
}
```

Field	Description
order_id	Store order ID
payment_status	Payment result (e.g. COD, success spelling used by store)
payment_type	e.g. <code>COD</code> , <code>online</code>
total	Order total before final adjustments as stored
final_amount	Amount charged / discounted total used by Comez
shipping_price	Shipping charged
total_discounted	Total discount applied
customer	Buyer details (<code>id</code> may be <code>null</code> for guests)
items	Line items
created_at	Order created time

cart.abandoned — data

```
{
  "customer_id": 42,
  "cart_token": null,
  "is_guest": false,
  "reason": "cart inactive for more than 10 days",
  "products": [
    {
      "cart_id": 101,
      "product_id": 1,
      "variant_id": 2,
      "quantity": 1
    }
  ],
  "abandoned_at": "2026-07-15T12:00:00.000Z"
}
```

Field	Description
customer_id	Registered customer ID, or <code>null</code> for guests
cart_token	Guest/session cart token when applicable
is_guest	Whether this was a guest cart
reason	Why the cart was marked abandoned
products	Items left in the cart
abandoned_at	When the webhook was generated

5. Verify the signature (required)

Comez signs the **exact raw request body** with your subscription secret:

```
HMAC-SHA256(secret, raw_body) → hex
Header: X-Comez-Signature: sha256=<hex>
```

Node.js (Express)

```
import crypto from "crypto";
import express from "express";

const app = express();
```

```
function verifyComezSignature(secret, rawBody, signatureHeader) {
  const expected = crypto
    .createHmac("sha256", secret)
    .update(rawBody)
    .digest("hex");

  const received = String(signatureHeader || "").replace(/^sha256=/i, "");
  if (!received || received.length !== expected.length) return false;

  return crypto.timingSafeEqual(
    Buffer.from(received),
    Buffer.from(expected)
  );
}

// Important: use raw body for this route (do not JSON-parse first)
app.post(
  "/webhooks/comez",
  express.raw({ type: "application/json" }),
  (req, res) => {
    const ok = verifyComezSignature(
      process.env.COMEZ_WEBHOOK_SECRET,
      req.body,
      req.headers["x-comez-signature"]
    );

    if (!ok) {
      return res.status(401).send("Invalid signature");
    }

    const payload = JSON.parse(req.body.toString("utf8"));

    // Process asynchronously if needed, but respond quickly
    // switch (payload.event) { ... }

    return res.status(200).send("ok");
  }
);
```

Python

```
import hmac
import hashlib

def verify_comez_signature(secret: str, raw_body: bytes, signature_header: str) -> bool:
    expected = hmac.new(
        secret.encode("utf-8"),
        raw_body,
        hashlib.sha256,
    ).hexdigest()
    received = (signature_header or "").removeprefix("sha256=")
    return hmac.compare_digest(expected, received)
```

Tips

- Always verify against the **raw bytes**, not a re-serialized JSON object.
- Use a constant-time compare (`timingSafeEqual` / `compare_digest`).
- Reject requests that fail verification with `401` .

6. Your endpoint requirements

Requirement	Detail
Protocol	HTTPS in production
Method	<code>POST</code>
Success	Return 2xx (ideally <code>200</code>)
Timeout	Respond within 5 seconds
Idempotency	Use <code>delivery_id</code> / <code>X-Comez-Delivery-Id</code> so retries are safe
Body size	Keep processing light; queue heavy work after acknowledging

If Comez does not receive a 2xx, the delivery is treated as failed and retried.

7. Retries and reliability

Setting	Value
Max attempts per delivery	6
Backoff	Exponential, starting at 30 seconds
HTTP timeout	5 seconds
Auto-disable	After 19 consecutive failed delivery jobs, the subscription is set inactive

- If a subscription is auto-disabled:
1. Fix your endpoint (uptime, SSL, 2xx responses, signature handling).
 2. Re-enable the webhook in Comez admin.

A successful delivery resets the failure counter.

8. Security checklist

- ☐ Store the signing secret securely (env var / secret manager).
- ☐ Never log the full secret.
- ☐ Verify `X-Comez-Signature` on every request.
- ☐ Prefer HTTPS only.
- ☐ Treat payloads as untrusted input until signature checks pass.
- ☐ Deduplicate using `delivery_id` if your handler is not idempotent.

9. Testing

1. Create a subscription pointing to a temporary inspector (e.g. webhook.site).
2. Place a test order (or trigger an abandoned-cart scenario).
3. Confirm headers and body.
4. Switch to your real endpoint and verify signature validation.

10. FAQ

When is `order.received` sent for online payments?

Only after payment verification succeeds — not when the unpaid order draft is created.

Can guest orders have `customer.id = null`?

Yes. Use `customer.name`, `email`, and `phone` from the payload.

Why am I not receiving events?

Check: subscription is **Active**, URL is reachable over HTTPS, endpoint returns 2xx, and the event type matches (`order.received` vs `cart.abandoned`).

Is the secret returned later?

No. It is shown only once at creation. If lost, delete the subscription and create a new one.

Support

If deliveries fail repeatedly or your subscription was auto-disabled, contact your Comez store admin with:

- Subscription ID / event type
- Approximate order ID or time
- Your endpoint status codes and signature verification logs (never share the raw secret)